

Bitfilters

Compact membership checks for large sets.



Install & load

```
INSTALL bitfilters FROM community;  
LOAD bitfilters;
```

01 At a glance

Aggregate hashed keys into a compact BLOB, then probe it to reject definite nonmembers. Use an exact check for candidates that survive the filter.

Hash keys



Build filter



Probe & verify

02 Core SQL surface

`xor8_filter(hash)`

Aggregate UBIGINT hashes into a static XOR filter.

`xor8_filter_contains(filter, hash)`

Return whether a key might be in the set.

`xor16_filter` / `xor16_filter_contains`

Use wider fingerprints for fewer false positives.

`binary_fuse8_filter` / `binary_fuse16_filter`

Alternative static filters with matching `_contains` probes.

`quotient_filter(q, r, hash)`

Build a quotient filter; q and r control capacity and precision.

`quotient_filter_contains(filter, hash)`

Probe a quotient filter with the same hash representation.

03 Keep in mind

A positive is only a candidate

Keep an exact join or membership predicate after probing. False positives are expected; correctly built filters retain inserted keys.

04 Build once

```
CREATE TABLE known_filter AS  
SELECT xor8_filter(hash(i)) AS f  
FROM range(1000) AS t(i);
```

Constructors aggregate rows; the result is a BLOB you can persist.

05 Probe candidates

```
SELECT i, xor8_filter_contains(f, hash(i))  
AS maybe_known  
FROM known_filter, range(995, 1005) AS t(i);
```

Inserted keys return true. An unseen key can also return true.

06 Confirm the match

```
SELECT i  
FROM known_filter, range(995, 1005) AS t(i)  
WHERE xor8_filter_contains(f, hash(i))  
AND i IN (SELECT range FROM range(1000));
```

The final exact predicate returns only IDs 995 through 999.

07 In practice

Hash consistently

Use the same function, seed and SQL input type when building and probing. Treat DuckDB `hash()` as version-dependent; persist its version or choose an explicit hash algorithm.

[Full docs & examples ↗](#) [Source ↗](#)